

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:   Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPI      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH      4
36 // #define TX_MSG_OBJ_ID          0 //transmit
37
38 #define RX_MSG_DATA_LENGTH      8
39 #define RX_MSG_OBJ_ID_1        1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2        2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3        3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4        4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADCC_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57
58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // ----- code for CAN start here -----
64 // volatile uint32_t txMsgCount = 0;
65 // extern uint16_t txMsgData[4];
66
67 volatile uint32_t rxMsgCount_1 = 0;
68 volatile uint32_t rxMsgCount_2 = 0;
69 extern uint16_t rxMsgData[8];
70
71 uint32_t dis_raw_1[2];
72 uint32_t dis_raw_2[2];
73 uint32_t dis_1 = 0;
74 uint32_t dis_2 = 0;
75
76 uint32_t quality_raw_1[4];
77 uint32_t quality_raw_2[4];
78 float quality_1 = 0.0;
79 float quality_2 = 0.0;
80
81 uint32_t lightlevel_raw_1[4];
82 uint32_t lightlevel_raw_2[4];
83 float lightlevel_1 = 0.0;
84 float lightlevel_2 = 0.0;
85
86 uint32_t measure_status_1 = 0;
87 uint32_t measure_status_2 = 0;
88
89 volatile uint32_t errorFlag = 0;
90 // ----- code for CAN end here -----
91
92 uint32_t numTimer0calls = 0;
93 uint16_t UARTPrint = 0;
94
95 float printLV1 = 0;
96 float printLV2 = 0;
97
98 float printLinux1 = 0;
99 float printLinux2 = 0;
100
101 uint16_t LADARi = 0;
102 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
103 uint16_t G_len = 11; //length of command
104 xy_ladar_pts[228]; //xy data
105 float LADARrightfront = 0;
106 float LADARfront = 0;
107 float LADARtemp_x = 0;
108 float LADARtemp_y = 0;
109 extern datapts_ladar_data[228];
110
111 extern uint16_t newLinuxCommands;
112 extern float LinuxCommands[CMDNUM_FROM_FLOATS];

```

```

113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1; // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2; // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1; // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 uint32_t numThres1 = 0;
161 uint32_t numThres2 = 0;
162
163 pose ROBOTps = {0,0,0}; //robot position
164 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
165 float LADARxoffset = 0;
166 float LADARyoffset = 0;
167
168 uint32_t timecount = 0;
169 int16_t RobotState = 1;
170 int16_t checkfronttally = 0;
171 int32_t WallFollowtime = 0;
172
173 #define NUMWAYPOINTS 8
174 uint16_t statePos = 0;
175 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
176 uint16_t i = 0; //for loop
177
178 uint16_t right_wall_follow_state = 2; // right follow
179 float Kp_front_wall = -2.0;
180 float front_turn_velocity = 0.2;
181 float left_turn_Stop_threshold = 3.5;
182 float Kp_right_wal = -4.0;
183 float ref_right_wall = 1.1;
184 float foward_velocity = 1.0;
185 float left_turn_Start_threshold = 1.3;
186 float turn_saturation = 2.5;
187
188 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
189
190 //more kalman vars
191 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
192 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
193 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
194 float z[3][1]; // state measurement
195 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
196 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
197 #define ProcUncert 0.0001
198 #define CovScalar 10
199 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
200 {0,ProcUncert,ProcUncert/CovScalar},
201 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
202 #define MeasUncert 1
203 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
204 {0,MeasUncert,MeasUncert/CovScalar},
205 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of Optitrack Motion Capture measurement)
206 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
207 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
208 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
209 float temp_3x3[3][3]; // intermediate storage
210 float temp_3x1[3][1]; // intermediate storage
211 float ytilde[3][1]; // difference between predictions
212
213 // Optitrack Variables
214 int32_t OptiNumUpdatesProcessed = 0;
215 pose OPTITRACKps;
216 extern uint16_t new_optitrack;
217 extern float Optitrackdata[OPTITRACKDATASIZE];
218 int16_t newOPTITRACKpose=0;
219
220 int16_t adcc2result = 0;
221 int16_t adcc3result = 0;
222 int16_t adcc4result = 0;
223 int16_t adcc5result = 0;
224 float adcc2Volt = 0.0;
225 float adcc3Volt = 0.0;
226 float adcc4Volt = 0.0;

```

```

227 float adc5Volt = 0.0;
228 int32_t numADCCalls = 0;
229 float LeftWheel = 0;
230 float RightWheel = 0;
231 float LeftWheel_1 = 0;
232 float RightWheel_1 = 0;
233 float LeftVel = 0;
234 float RightVel = 0;
235 float uLeft = 0;
236 float uRight = 0;
237 float HandValue = 0;
238 float vref = 0;
239 float turn = 0;
240 float gyro9250_drift = 0;
241 float gyro9250_angle = 0;
242 float old_gyro9250 = 0;
243 float gyroLPR510_angle = 0;
244 float gyroLPR510_offset = 0;
245 float gyroLPR510_drift = 0;
246 float old_gyroLPR510 = 0;
247 float gyro9250_radians = 0;
248 float gyroLPR510_radians = 0;
249
250 int16_t readdata[25];
251 int16_t IMU_data[9];
252
253 float accelx = 0;
254 float accely = 0;
255 float accelz = 0;
256 float gyrox = 0;
257 float gyroy = 0;
258 float gyroz = 0;
259
260 // Needed global Variables
261 float accelx_offset = 0;
262 float accely_offset = 0;
263 float accelz_offset = 0;
264 float gyrox_offset = 0;
265 float gyroy_offset = 0;
266 float gyroz_offset = 0;
267 int16_t calibration_state = 0;
268 int32_t calibration_count = 0;
269 int16_t doneCal = 0;
270
271 #define MPU9250 1
272 #define DAN28027 2
273 int16_t CurrentChip = MPU9250;
274 int16_t DAN28027Garbage = 0;
275 int16_t dan28027adc1 = 0;
276 int16_t dan28027adc2 = 0;
277 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
278
279 void main(void)
280 {
281     // PLL, WatchDog, enable Peripheral Clocks
282     // This example function is found in the F2837xD_SysCtrl.c file.
283     InitSysCtrl();
284
285     InitGpio();
286
287     InitSE423DefaultGPIO();
288
289     //Scope for Timing
290     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
291     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
292     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
293
294     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
295     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
296     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
297
298     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
299     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
300     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
301
302     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
303     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
304     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
305
306     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
307     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
308     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
309
310     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
311     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
312     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
313
314 // ----- code for CAN start here -----
315 //GPIO17 - CANRXB
316 GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
317 GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
318
319 //GPIO12 - CANTXB
320 GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
321 GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
322 // ----- code for CAN end here -----
323
324
325
326 // ----- code for CAN start here -----
327 // Initialize the CAN controller
328 InitCANB();
329
330 // Set up the CAN bus bit rate to 1000 kbps
331 setCANBitRate(200000000, 1000000);
332
333 // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
334 CanbRegs.CAN_CTL.bit.IE0= 1;
335 CanbRegs.CAN_CTL.bit.EIE= 1;
336 // ----- code for CAN end here -----
337
338 // Clear all interrupts and initialize PIE vector table:
339 // Disable CPU interrupts
340 DINT;

```

```

341 // Initialize the PIE control registers to their default state.
342 // The default state is all PIE interrupts disabled and flags
343 // are cleared.
344 // This function is found in the F2837xD_PieCtrl.c file.
345 InitPieCtrl();
346
347 // Disable CPU interrupts and clear all CPU interrupt flags:
348 IER = 0x0000;
349 IFR = 0x0000;
350
351 // Initialize the PIE vector table with pointers to the shell Interrupt
352 // Service Routines (ISR).
353 // This will populate the entire table, even if the interrupt
354 // is not used in this example. This is useful for debug purposes.
355 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
356 // This function is found in F2837xD_PieVect.c.
357 InitPieVectTable();
358
359 // Interrupts that are used in this example are re-mapped to
360 // ISR functions found within this project
361 EALLOW; // This is needed to write to EALLOW protected registers
362 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
363 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
364 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
365 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
366 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
367 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
368 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
369 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
370 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
371 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
372 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
373 PieVectTable.ADCCL1_INT = &ADCC_ISR;
374 PieVectTable.SPIB_RX_INT = &SPIB_isr;
375 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
376 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
377 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
378 // ----- code for CAN start here -----
379 PieVectTable.CANB0_INT = &can_isr;
380 // ----- code for CAN end here -----
381 EDIS; // This is needed to disable write to EALLOW protected registers
382
383 // Initialize the CpuTimers Device Peripheral. This function can be
384 // found in F2837xD_CpuTimers.c
385 InitCpuTimers();
386
387 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
388 // 200MHz CPU Freq, Period (in uSeconds)
389 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
390 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
391 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
392
393 // Enable CpuTimer Interrupt bit TIE
394 CpuTimer0Regs.TCR.all = 0x4000;
395 CpuTimer1Regs.TCR.all = 0x4000;
396 CpuTimer2Regs.TCR.all = 0x4000;
397
398 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
399
400 init_serialSCIB(&SerialB,19200);
401 init_serialSCIC(&SerialC,19200);
402
403 for (LADARi = 0; LADARi < 228; LADARi++) {
404     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
405 }
406
407 init_eQEPs();
408 init_EPWM1and2();
409 init_RCServoPWM_3AB_5AB_6A();
410 init_ADCsAndDACs();
411 setupSpib();
412
413 EALLOW;
414 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOCA on A group
415 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
416 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
417 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
418 EPwm4Regs.TBCTR = 0x0; // Clear counter
419 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
420 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
421 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
422 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
423 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
424 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
425 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
426 EDIS;
427
428 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
429 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
430 // to CPU-Timer 2: int 12 is for the SWI.
431 IER |= M_INT1;
432 IER |= M_INT6;
433 IER |= M_INT8; // SCIC SCID
434 IER |= M_INT9; // SCIA
435 IER |= M_INT12;
436 IER |= M_INT13;
437 IER |= M_INT14;
438
439 // Enable TINT0 in the PIE: Group 1 interrupt 7
440 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
441 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
442 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
443 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
444 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
445 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
446 // ----- code for CAN start here -----
447 // Enable CANB in the PIE: Group 9 interrupt 7
448 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
449 // ----- code for CAN end here -----
450 // ----- code for CAN start here -----
451 // Enable the CAN interrupt signal
452 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;

```

```

455 // ----- code for CAN end here -----
456
457
458 robotdest[0].x = -4;    robotdest[0].y = 10;
459 robotdest[1].x = -4;    robotdest[1].y = 2;
460 //middle of bottom
461 robotdest[2].x = 0;      robotdest[2].y = 2;
462 //outside the course
463 robotdest[3].x = 0;      robotdest[3].y = -3;
464 //back to middle
465 robotdest[4].x = 0;      robotdest[4].y = 2;
466 robotdest[5].x = 4;      robotdest[5].y = 2;
467 robotdest[6].x = 4;      robotdest[6].y = 10;
468 robotdest[7].x = 0;      robotdest[7].y = 9;
469
470 // ROBOTps will be updated by Optitrack during gyro calibration
471 // TODO: specify the starting position of the robot
472 ROBOTps.x = 0;           //the estimate in array form (useful for matrix operations)
473 ROBOTps.y = 0;
474 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
475 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
476 x_pred[1][0] = ROBOTps.y;
477 x_pred[2][0] = ROBOTps.theta;
478
479 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
480 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
481 SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
482 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
483 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
484 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
485
486 init_serialSCIA(&SerialA,115200);
487 init_serialSCID(&SerialD,2083332);
488
489 // Enable global Interrupts and higher priority real-time debug events
490 EINT; // Enable Global interrupt INTM
491 ERTM; // Enable Global realtime interrupt DBGM
492 // ----- code for CAN start here -----
493 // Measured Distance from 1
494 // Initialize the receive message object 1 used for receiving CAN messages.
495 // Message Object Parameters:
496 //     Message Object ID Number: 1
497 //     Message Identifier: 0x060b0101
498 //     Message Frame: Standard
499 //     Message Type: Receive
500 //     Message ID Mask: 0x0
501 //     Message Object Flags: Receive Interrupt
502 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
503 //     for a Receive mailbox)
504 //
505 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
506                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
507                       RX_MSG_DATA_LENGTH);
508
509 // Measured Distance from 2
510 // Initialize the receive message object 2 used for receiving CAN messages.
511 // Message Object Parameters:
512 //     Message Object ID Number: 2
513 //     Message Identifier: 0x060b0102
514 //     Message Frame: Standard
515 //     Message Type: Receive
516 //     Message ID Mask: 0x0
517 //     Message Object Flags: Receive Interrupt
518 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
519 //     for a Receive mailbox)
520 //
521 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
522                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
523                       RX_MSG_DATA_LENGTH);
524
525 // Measurement Quality from 1
526 // Initialize the receive message object 2 used for receiving CAN messages.
527 // Message Object Parameters:
528 //     Message Object ID Number: 3
529 //     Message Identifier: 0x060b0201
530 //     Message Frame: Standard
531 //     Message Type: Receive
532 //     Message ID Mask: 0x0
533 //     Message Object Flags: Receive Interrupt
534 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
535 //     for a Receive mailbox)
536 //
537 //
538 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
539                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
540                       RX_MSG_DATA_LENGTH);
541
542 // Measurement Quality from 2
543 // Initialize the receive message object 2 used for receiving CAN messages.
544 // Message Object Parameters:
545 //     Message Object ID Number: 4
546 //     Message Identifier: 0x060b0202
547 //     Message Frame: Standard
548 //     Message Type: Receive
549 //     Message ID Mask: 0x0
550 //     Message Object Flags: Receive Interrupt
551 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
552 //     for a Receive mailbox)
553 //
554 //
555 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
556                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
557                       RX_MSG_DATA_LENGTH);
558
559
560 //
561 // Start CAN module operations
562 //
563 CanbRegs.CAN_CTL.bit.Init = 0;
564 CanbRegs.CAN_CTL.bit.CCE = 0;
565
566
567 // ----- code for CAN end here -----
568 char S_command[19] = "S1152000124000\n"; //this change the baud rate to 115200

```

```

569     uint16_t S_len = 19;
570     serial_sendSCIC(&SerialC, S_command, S_len);
571
572
573     DELAY_US(1000000); // Delay letting Lidar change its Baud rate
574     init_serialSCIC(&SerialC, 115200);
575
576
577     // LED1 is GPIO22
578     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
579     // LED2 is GPIO94
580     GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
581     // LED3 is GPIO95
582     GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
583     // LED4 is GPIO97
584     GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
585     // LED5 is GPIO111
586     GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
587
588
589     // IDLE loop. Just sit and loop forever (optional):
590     while(1)
591     {
592         if (UARTPrint == 1) {
593
594             if (readbuttons() == 0) {
595                 UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
596                 // UART_printfLine(1, "x:%.2f y:%.2f a:%.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
597                 UART_printfLine(2, "F%.4f R%.4f", LADARfront, LADARrightfront);
598             } else if (readbuttons() == 1) {
599                 UART_printfLine(1, "01A:%.0fC:%.0fR:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
600                 UART_printfLine(2, "P1A:%.0fC:%.0fR:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
601                 //UART_printfLine(1, "LV1:%.3f LV2:%.3f", printLV1, printLV2);
602                 //UART_printfLine(2, "Ln1:%.3f Ln2:%.3f", printLinux1, printLinux2);
603             } else if (readbuttons() == 2) {
604                 UART_printfLine(1, "02A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
605                 UART_printfLine(2, "P2A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
606                 // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
607                 // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
608             } else if (readbuttons() == 4) {
609                 UART_printfLine(1, "03A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
610                 UART_printfLine(2, "P3A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
611                 // UART_printfLine(1, "L:%.3f R:%.3f", LeftVel, RightVel);
612                 // UART_printfLine(2, "uL:%.2f uR:%.2f", uLeft, uRight);
613             } else if (readbuttons() == 8) {
614                 UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
615                 UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
616             } else if (readbuttons() == 3) {
617                 UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
618                 UART_printfLine(2, "MPU:%.2f LPR:%.2f", gyro9250_radians, gyroLPR510_radians);
619             } else if (readbuttons() == 5) {
620                 UART_printfLine(1, "0x:%.2f 0y:%.2f 0a:%.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
621                 UART_printfLine(2, "State:%d : %d", RobotState, statePos);
622             } else if (readbuttons() == 6) {
623                 UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
624                 UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
625             } else if (readbuttons() == 7) {
626                 UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
627                 UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
628             }
629
630             UARTPrint = 0;
631         }
632     }
633 }
634
635 __interrupt void SPIB_isr(void){
636
637     uint16_t i;
638     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
639
640     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
641     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
642
643     if (CurrentChip == MPU9250) {
644
645         for (i=0; i<8; i++) {
646             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
647         }
648
649         PostSWI1(); // Manually cause the interrupt for the SWI1
650
651     } else if (CurrentChip == DAN28027) {
652
653         DAN28027Garbage = SpibRegs.SPIRXBUF;
654         dan28027adc1 = SpibRegs.SPIRXBUF;
655         dan28027adc2 = SpibRegs.SPIRXBUF;
656         CurrentChip = MPU9250;
657         SpibRegs.SPIFFCT.bit.TXDLY = 0;
658     }
659
660     SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
661     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
662     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
663     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
664 }
665
666
667 //adcd1 pie interrupt
668 __interrupt void ADCC_ISR (void)
669 {
670     GpioDataRegs.GPASET.bit.GPIO11 = 1;
671     adcc2result = AdccResultRegs.ADCRESULT0;
672     adcc3result = AdccResultRegs.ADCRESULT1;
673     adcc4result = AdccResultRegs.ADCRESULT2;
674     adcc5result = AdccResultRegs.ADCRESULT3;
675
676     // Here covert ADCIND0 to volts
677     adcC2Volt = adcc2result*3.0/4095.0;
678     adcC3Volt = adcc3result*3.0/4095.0;
679     adcC4Volt = adcc4result*3.0/4095.0;
680     adcC5Volt = adcc5result*3.0/4095.0;
681
682     if (MPU9250ignoreCNT >= 1) {

```



```

683     CurrentChip = MPU9250;
684     SpibRegs.SPIFFCT.bit.TXDLY = 0;
685     SpibRegs.SPIFFRX.bit.RXFFIL = 8;
686
687     GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
688
689     SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
690     SpibRegs.SPITXBUF = 0;
691     SpibRegs.SPITXBUF = 0;
692     SpibRegs.SPITXBUF = 0;
693     SpibRegs.SPITXBUF = 0;
694     SpibRegs.SPITXBUF = 0;
695     SpibRegs.SPITXBUF = 0;
696     SpibRegs.SPITXBUF = 0;
697 } else {
698     MPU9250ignoreCNT++;
699 }
700
701 numADCCalls++;
702 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
703 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
704 GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
705 }
706 }
707
708 // cpu_timer0_isr - CPU Timer0 ISR
709 __interrupt void cpu_timer0_isr(void)
710 {
711     CpuTimer0.InterruptCount++;
712
713     numTimer0calls++;
714
715     if ((numTimer0calls%5) == 0) {
716         // Blink LaunchPad Red LED
717         //GpioDataRegs.GPBTGGLE.bit.GPIO34 = 1;
718     }
719
720     // Acknowledge this interrupt to receive more interrupts from group 1
721     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
722 }
723
724 // cpu_timer1_isr - CPU Timer1 ISR
725 __interrupt void cpu_timer1_isr(void)
726 {
727
728     serial_sendSCIC(&SerialC, G_command, G_len);
729
730     CpuTimer1.InterruptCount++;
731 }
732
733 // cpu_timer2_isr CPU Timer2 ISR
734 __interrupt void cpu_timer2_isr(void)
735 {
736     // Blink LaunchPad Blue LED
737     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
738
739     CpuTimer2.InterruptCount++;
740
741     // if ((CpuTimer2.InterruptCount % 10) == 0) {
742     //     UARTPrint = 1;
743     // }
744 }
745
746 void setF28027EPWM1A(float controleffort){
747     if (controleffort < -10) {
748         controleffort = -10;
749     }
750     if (controleffort > 10) {
751         controleffort = 10;
752     }
753     float value = (controleffort+10)*3000.0/20.0;
754     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
755 }
756 void setF28027EPWM2A(float controleffort){
757     if (controleffort < -10) {
758         controleffort = -10;
759     }
760     if (controleffort > 10) {
761         controleffort = 10;
762     }
763     float value = (controleffort+10)*3000.0/20.0;
764     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
765 }
766
767 //
768 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
769 //
770 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
771 {
772     GpioDataRegs.GPASET.bit.GPIO61 = 1;
773     // Set interrupt priority:
774     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
775     IER |= M_INT12;
776     IER &= MINT12; // Set "global" priority
777     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
778     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
779     __asm(" NOP");
780     EINT;
781
782     //#####
783     IMU_data[0] = readdata[1];
784     IMU_data[1] = readdata[2];
785     IMU_data[2] = readdata[3];
786     IMU_data[3] = readdata[5];
787     IMU_data[4] = readdata[6];
788     IMU_data[5] = readdata[7];
789
790     accelx = (((float)(IMU_data[0]))*4.0/32767.0);
791     accely = (((float)(IMU_data[1]))*4.0/32767.0);
792     accelz = (((float)(IMU_data[2]))*4.0/32767.0);
793     gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
794     gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
795     gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
796

```

```

797     if(calibration_state == 0){
798         calibration_count++;
799         if (calibration_count == 2000) {
800             calibration_state = 1;
801             calibration_count = 0;
802         }
803     } else if(calibration_state == 1){
804         accelx_offset+=accelx;
805         accely_offset+=accely;
806         accelz_offset+=accelz;
807         gyro_x_offset+=gyrox;
808         gyro_y_offset+=gyroy;
809         gyro_z_offset+=gyroz;
810         gyroLPR510_offset+=adcC5Volt;
811         calibration_count++;
812         if (calibration_count == 2000) {
813             calibration_state = 2;
814             accelx_offset/=2000.0;
815             accely_offset/=2000.0;
816             accelz_offset/=2000.0;
817             gyro_x_offset/=2000.0;
818             gyro_y_offset/=2000.0;
819             gyro_z_offset/=2000.0;
820             gyroLPR510_offset/=2000.0;
821             calibration_count = 0;
822         }
823     } else if(calibration_state == 2) {
824
825         gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
826         old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
827
828         gyro9250_drift += (((gyroz-gyro_z_offset) + old_gyro9250)*.0005)/(2000);
829         old_gyro9250 = gyro_z-gyro_z_offset;
830         calibration_count++;
831
832         if (calibration_count == 2000) {
833             calibration_state = 3;
834             calibration_count = 0;
835             doneCal = 1;
836             newOPTITRACKpose = 0;
837         }
838     } else if(calibration_state == 3){
839         accelx -=(accelx_offset);
840         accely -=(accely_offset);
841         accelz -=(accelz_offset);
842         gyro_x -= gyro_x_offset;
843         gyro_y -= gyro_y_offset;
844         gyro_z -= gyro_z_offset;
845         LeftWheel = readEncLeft();
846         RightWheel = readEncRight();
847         HandValue = readEncWheel();
848
849         gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
850         old_gyro9250 = gyro_z;
851
852         gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
853         old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
854
855         gyro9250_radians = (gyro9250_angle * (PI/180.0));
856         gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
857
858         LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
859         RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
860
861         if (newLinuxCommands == 1) {
862             newLinuxCommands = 0;
863             printLinux1 = LinuxCommands[0];
864             printLinux2 = LinuxCommands[1];
865             ///? = LinuxCommands[2];
866             ///? = LinuxCommands[3];
867             ///? = LinuxCommands[4];
868             ///? = LinuxCommands[5];
869             ///? = LinuxCommands[6];
870             ///? = LinuxCommands[7];
871             ///? = LinuxCommands[8];
872             ///? = LinuxCommands[9];
873             ///? = LinuxCommands[10];
874         }
875
876         if (NewCAMDataThreshold1 == 1) {
877             NewCAMDataThreshold1 = 0;
878             MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
879             MaxColThreshold1 = fromCAMvaluesThreshold1[1];
880             MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
881
882             NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
883             NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
884             NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
885
886             NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
887             NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
888             NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
889             numThres1++;
890             if ((numThres1 % 5) == 0) {
891                 // LED4 is GPIO97
892                 GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
893             }
894         }
895
896         if (NewCAMDataThreshold2 == 1) {
897             NewCAMDataThreshold2 = 0;
898             MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
899             MaxColThreshold2 = fromCAMvaluesThreshold2[1];
900             MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
901
902             NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
903             NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
904             NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
905
906             NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
907             NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
908             NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
909             numThres2++;
910             if ((numThres2 % 5) == 0) {

```



```

911         // LED5 is GPIO111
912         GpioDataRegs.GPDT0GGLE.bit.GPIO111 = 1;
913     }
914 }
915
916 if (NewCAMDataAprilTag1 == 1) {
917     NewCAMDataAprilTag1 = 0;
918     tagx = fromCAMvaluesAprilTag1[0];
919     tagy = fromCAMvaluesAprilTag1[1];
920     tagz = fromCAMvaluesAprilTag1[2];
921
922     tagthetax = fromCAMvaluesAprilTag1[3];
923     tagthetay = fromCAMvaluesAprilTag1[4];
924     tagthetaz = fromCAMvaluesAprilTag1[5];
925
926     tagid = fromCAMvaluesAprilTag1[6];
927
928     numtag++;
929     if ((numtag % 5) == 0) {
930         // LED2 is GPIO94
931         GpioDataRegs.GPCT0GGLE.bit.GPIO94 = 1;
932     }
933 }
934
935 if (NewLVData == 1) {
936     NewLVData = 0;
937     printLV1 = fromLVvalues[0];
938     printLV2 = fromLVvalues[1];
939     ///? = fromLVvalues[2];
940     ///? = fromLVvalues[3];
941     ///? = fromLVvalues[4];
942     ///? = fromLVvalues[5];
943     ///? = fromLVvalues[6];
944     ///? = fromLVvalues[7];
945 }
946
947
948 if (new_optitrack == 1) {
949     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
950     new_optitrack = 0;
951 }
952
953 // Step 0: update B, u
954 B[0][0] = cosf(ROBOTps.theta)*0.001;
955 B[1][0] = sinf(ROBOTps.theta)*0.001;
956 B[2][1] = 0.001;
957 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
958 u[1][0] = gyroz*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
959
960 // Step 1: predict the state and estimate covariance
961 Matrix3x2_Mult(B, u, Bu); // Bu = B*u
962 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
963 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
964 // Step 2: if there is a new measurement, then update the state
965 if (1 == newOPTITRACKpose) {
966     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
967     if ((OptiNumUpdatesProcessed%10)==0) {
968         // LED 3
969         GpioDataRegs.GPCT0GGLE.bit.GPIO95 = 1;
970     }
971     newOPTITRACKpose = 0;
972     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
973     z[1][0] = OPTITRACKps.y;
974     // fix for OptiTrack problem at 180 degrees
975     if (cosf(ROBOTps.theta) < -0.99) {
976         z[2][0] = ROBOTps.theta;
977     }
978     else {
979         z[2][0] = OPTITRACKps.theta;
980     }
981     // Step 2a: calculate the innovation/measurement residual, ytilde
982     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
983     // Step 2b: calculate innovation covariance, S
984     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
985     // Step 2c: calculate the optimal Kalman gain, K
986     Matrix3x3_Invert(S, S_inv);
987     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
988     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
989     Matrix3x1_Mult(K, ytilde, temp_3x1);
990     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
991     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
992     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
993     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
994 } // end of correction step
995
996 // set ROBOTps to the updated and corrected Kalman values.
997 ROBOTps.x = x_pred[0][0];
998 ROBOTps.y = x_pred[1][0];
999 ROBOTps.theta = x_pred[2][0];
1000
1001 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1002 // uses xy code to step through an array of positions
1003 if (xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1004     statePos = (statePos+1)%NUMWAYPOINTS;
1005 }
1006 // state machine
1007 switch (RobotState) {
1008 case 1:
1009
1010     // vref and turn are the vref and turn returned from xy_control
1011
1012     if (LADARfront < 1.2) {
1013         vref = 0.2;
1014         checkfronttally++;
1015         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1016             RobotState = 10; // Wall follow
1017             WallFollowtime = 0;
1018             right_wall_follow_state = 1;
1019         }
1020     } else {
1021         checkfronttally = 0;
1022     }
1023
1024     break;

```

```

1025     case 10:
1026         if (right_wall_follow_state == 1) {
1027             //Left Turn
1028             turn = Kp_front_wall*(14.5 - LADARfront);
1029             vref = front_turn_velocity;
1030             if (LADARfront > left_turn_Stop_threshold) {
1031                 right_wall_follow_state = 2;
1032             }
1033         } else if (right_wall_follow_state == 2) {
1034             //Right Wall Follow
1035             turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1036             vref = foward_velocity;
1037             if (LADARfront < left_turn_Start_threshold) {
1038                 right_wall_follow_state = 1;
1039             }
1040         }
1041         if (turn > turn_saturation) {
1042             turn = turn_saturation;
1043         }
1044         if (turn < -turn_saturation) {
1045             turn = -turn_saturation;
1046         }
1047
1048         WallFollowtime++;
1049         if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1050             RobotState = 1;
1051             checkfronttally = 0;
1052         }
1053         break;
1054
1055     case 20:
1056         // put vision code here
1057         break;
1058     default:
1059         break;
1060 }
1061
1062
1063 //Must be called each time into this SWI1 function
1064 PIcontrl(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1065 // These below lines also must be called each time into this SWI1 function
1066 setEPWM1A(uLeft);
1067 setEPWM2A(-uRight);
1068 setF28027EPWM1A(uLeft);
1069 setF28027EPWM2A(-uRight);
1070 LeftWheel_1 = LeftWheel;
1071 RightWheel_1 = RightWheel;
1072
1073 if((timecount%250) == 0) {
1074     DataToLabView.floatData[0] = ROBOTps.x;
1075     DataToLabView.floatData[1] = ROBOTps.y;
1076     DataToLabView.floatData[2] = ROBOTps.theta;
1077     DataToLabView.floatData[3] = (float)timecount;
1078     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1079     DataToLabView.floatData[5] = (float)RobotState;
1080     DataToLabView.floatData[6] = (float)statePos;
1081     DataToLabView.floatData[7] = LADARfront;
1082     LVsenddata[0] = '*'; // header for LVdata
1083     LVsenddata[1] = '$';
1084     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1085         if (i%2==0) {
1086             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1087         } else {
1088             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1089         }
1090     }
1091     serial_sendSCID(&SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1092 }
1093
1094 }
1095 timecount++;
1096 if((timecount%200) == 0)
1097 {
1098     if(doneCal == 0) {
1099         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1100     }
1101     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1102     UARTPrint = 1; // Tell While loop to print
1103 }
1104
1105 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1106 CurrentChip = DAN28027;
1107 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1108 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1109 SpibRegs.SPITXBUF = 0x00DA;
1110 SpibRegs.SPITXBUF = EPwm1A_F28027;
1111 SpibRegs.SPITXBUF = EPwm2A_F28027;
1112
1113
1114 //#####
1115 //
1116 // Restore registers saved:
1117 //
1118 DINT;
1119 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1120
1121 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1122 }
1123
1124 //
1125 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1126 //
1127 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1128 {
1129     // Set interrupt priority:
1130     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1131     IER |= M_INT12;
1132     IER &= MINT12; // Set "global" priority
1133     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1134     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1135     __asm(" NOP");
1136     EINT;
1137
1138 //#####

```

```

1139 // Insert SWI ISR Code here.....
1140 // LED1
1141 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1142 if (LADARpingpong == 1) {
1143     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1144     LADARrightfront = 19; // 19 is greater than max feet
1145     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1146         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1147             LADARrightfront = ladar_data[LADARi].distance_ping;
1148         }
1149     }
1150     // LADARfront is the min of dist 111, 112, 113, 114, 115
1151     LADARfront = 19;
1152     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1153         if (ladar_data[LADARi].distance_ping < LADARfront) {
1154             LADARfront = ladar_data[LADARi].distance_ping;
1155         }
1156     }
1157     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
1158     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
1159     for (LADARi = 0; LADARi < 228; LADARi++) {
1160
1161         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1162         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1163     }
1164 }
1165 } else if (LADARpingpong == 0) {
1166     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1167     LADARrightfront = 19; // 19 is greater than max feet
1168     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1169         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1170             LADARrightfront = ladar_data[LADARi].distance_pong;
1171         }
1172     }
1173     // LADARfront is the min of dist 111, 112, 113, 114, 115
1174     LADARfront = 19;
1175     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1176         if (ladar_data[LADARi].distance_pong < LADARfront) {
1177             LADARfront = ladar_data[LADARi].distance_pong;
1178         }
1179     }
1180     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1181     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1182     for (LADARi = 0; LADARi < 228; LADARi++) {
1183
1184         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1185         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1186     }
1187 }
1188 }
1189
1190
1191
1192
1193 //#####
1194 //
1195 // Restore registers saved:
1196 //
1197 DINT;
1198 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1199
1200 }
1201
1202 //
1203 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1204 //
1205 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1206 {
1207     // Set interrupt priority:
1208     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1209     IER |= M_INT12;
1210     IER &= MINT12; // Set "global" priority
1211     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1212     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1213     __asm(" NOP");
1214     EINT;
1215
1216 //#####
1217 // Insert SWI ISR Code here.....
1218
1219 //#####
1220 //
1221 // Restore registers saved:
1222 //
1223 DINT;
1224 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1225
1226 }
1227
1228 // ----- code for CAN start here -----
1229 __interrupt void can_isr(void)
1230 {
1231     int i = 0;
1232
1233     uint32_t status;
1234
1235     //
1236     // Read the CAN interrupt status to find the cause of the interrupt
1237     //
1238     status = CANGetInterruptCause(CANB_BASE);
1239
1240     //
1241     // If the cause is a controller status interrupt, then get the status
1242     //
1243     if(status == CAN_INT_INT0ID_STATUS)
1244     {
1245         //
1246         // Read the controller status. This will return a field of status
1247         // error bits that can indicate various errors. Error processing
1248         // is not done in this example for simplicity. Refer to the
1249         // API documentation for details about the error status bits.
1250         // The act of reading this status will clear the interrupt.
1251         //
1252         status = CANgetStatus(CANB_BASE);

```

```

1253 }
1254
1255 //
1256 // Check if the cause is the receive message object 2
1257 //
1258 //
1259 else if(status == RX_MSG_OBJ_ID_1)
1260 {
1261     //
1262     // Get the received message
1263     //
1264     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1265
1266     for(i = 0; i<2; i++)
1267     {
1268         dis_raw_1[i] = rxMsgData[i];
1269     }
1270
1271     dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1272
1273     measure_status_1 = rxMsgData[2];
1274
1275     //
1276     // Getting to this point means that the RX interrupt occurred on
1277     // message object 2, and the message RX is complete. Clear the
1278     // message object interrupt.
1279     //
1280     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1281
1282     //
1283     // Increment a counter to keep track of how many messages have been
1284     // received. In a real application this could be used to set flags to
1285     // indicate when a message is received.
1286     //
1287     rxMsgCount_1++;
1288
1289     //
1290     // Since the message was received, clear any error flags.
1291     //
1292     errorFlag = 0;
1293 }
1294
1295 else if(status == RX_MSG_OBJ_ID_2)
1296 {
1297     //
1298     // Get the received message
1299     //
1300     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1301
1302     for(i = 0; i<2; i++)
1303     {
1304         dis_raw_2[i] = rxMsgData[i];
1305     }
1306
1307     dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1308
1309     measure_status_2 = rxMsgData[2];
1310
1311     //
1312     // Getting to this point means that the RX interrupt occurred on
1313     // message object 2, and the message RX is complete. Clear the
1314     // message object interrupt.
1315     //
1316     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1317
1318     //
1319     // Increment a counter to keep track of how many messages have been
1320     // received. In a real application this could be used to set flags to
1321     // indicate when a message is received.
1322     //
1323     rxMsgCount_2++;
1324
1325     //
1326     // Since the message was received, clear any error flags.
1327     //
1328     errorFlag = 0;
1329 }
1330
1331 else if(status == RX_MSG_OBJ_ID_3)
1332 {
1333     //
1334     // Get the received message
1335     //
1336     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1337
1338     for(i = 0; i<4; i++)
1339     {
1340         lightlevel_raw_1[i] = rxMsgData[i];
1341         quality_raw_1[i] = rxMsgData[i+4];
1342     }
1343
1344     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1345     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1346
1347     //
1348     // Getting to this point means that the RX interrupt occurred on
1349     // message object 2, and the message RX is complete. Clear the
1350     // message object interrupt.
1351     //
1352     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1353
1354     //
1355     // Since the message was received, clear any error flags.
1356     //
1357     errorFlag = 0;
1358 }
1359
1360 else if(status == RX_MSG_OBJ_ID_4)
1361 {
1362     //
1363     // Get the received message
1364     //
1365     //
1366

```

```

1367 //
1368 CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1369
1370 for(i = 0; i<4; i++)
1371 {
1372     lightlevel_raw_2[i] = rxMsgData[i];
1373     quality_raw_2[i] = rxMsgData[i+4];
1374 }
1375
1376 lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1377 quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1378
1379 //
1380 // Getting to this point means that the RX interrupt occurred on
1381 // message object 2, and the message RX is complete. Clear the
1382 // message object interrupt.
1383 //
1384 CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1385
1386 //
1387 // Since the message was received, clear any error flags.
1388 //
1389 errorFlag = 0;
1390 }
1391
1392
1393
1394
1395 //
1396 // If something unexpected caused the interrupt, this would handle it.
1397 //
1398 else
1399 {
1400     //
1401     // Spurious interrupt handling can go here.
1402     //
1403 }
1404
1405 //
1406 // Clear the global interrupt flag for the CAN interrupt line
1407 //
1408 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1409
1410 //
1411 // Acknowledge this interrupt located in group 9
1412 //
1413 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1414 }
1415 // ----- code for CAN end here -----

```